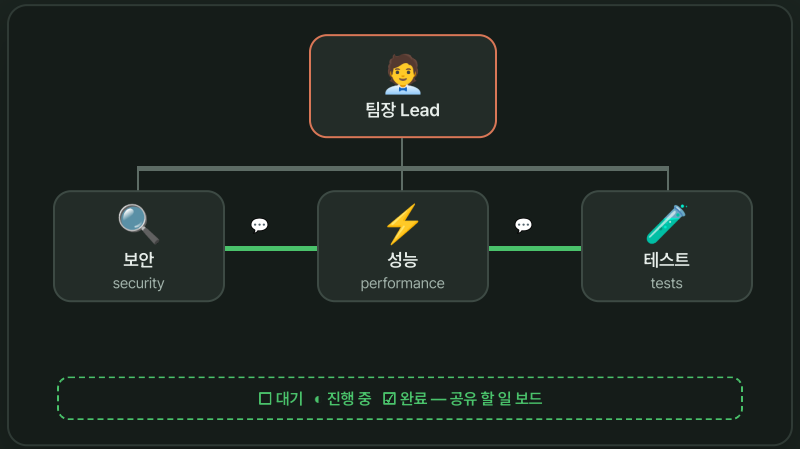
실습 자료 · 8시간

클로드 팀으로 도서 대출 앱 살리기

8시간 실습 · 단계마다 왜 하는지와 용어 풀이

Lead Teammates Task list



목차

들어가며	오늘의 실습 한눈에	3
읽는 법	이 실습 자료 읽는 법	4
시간표	시간표 한눈에 (09:00–18:00)	6
준비물	수업 전 준비물	8
블록 0	오리엔테이션 — 오늘의 환자 받기 (20분)	10
블록 1	심부름꾼(서브에이전트) 복습 (40분)	13
블록 2	팀 켜기 + 첫 팀 (30분)	21
블록 3	🔍 리뷰 팀 — 혼자 vs 팀 대결 (50분)	26
블록 4	💬 토론 + 직접 대화 (40분)	31
퀴즈	🧩 오전 복습 퀴즈 (20분)	35
블록 5	🛠️ 구현 팀 — 수리 + 예약 기능 (60분)	37
블록 6	🚧 검문소 — 테스트 못 넘으면 완료 금지 (40분)	42
블록 7	🕵️ 디버깅 토론 — 짝꿍이 심은 버그 찾기 (30분)	46
블록 8	🚀 나만의 팀 — 2바퀴 (90분)	50
블록 9	발표 & 회고 (30분)	53
부록 A	시간 조절	55
부록 B	자주 막히는 곳	56
부록 C	🔒 강사용 정답지 — 연습용 앱에 심어둔 문제	58

들어가며

오늘의 실습 한눈에

하루를 **두 바퀴**로 돕니다.

- **1바퀴 (09:00–15:50, 같이 따라하기)**: 일부러 문제를 심어둔 **도서 대출 앱**을 클로드 팀으로 리뷰 → 토론 → 고치기 → 기능 추가 → 디버깅까지
- **2바퀴 (16:00–18:00, 혼자 해보기)**: **내가 고른 상황**에 맞게 팀을 직접 설계하고 굴린다

핵심 원칙: **팀은 비싸다. "왜 팀이어야 하지?"에 답할 수 있을 때만 꾸린다.** 교재: 01-팀에이전트-쉬운요약.pdf · 연습용 앱: github.com/NujabesCode/library-app

읽는 법

이 실습 자료 읽는 법

입력하는 곳이 두 군데입니다

표시	어디에 입력?	예
⌨️ **터미널**	검은 명령 창 (Git Bash, 터미널). `$` 뒤에 치는 곳	`cd library-app`, `npm test`, `git commit`
💬 **클로드 대화창**	터미널에서 `claude` 를 실행한 **뒤** 나오는 입력창 (`>` 표시)	"Explore 서브에이전트로 ~ 해줘" 같은 **부탁 문장**

> 💬 상자의 문장은 **명령어가 아니라 클로드에게 하는 말**입니다. 그대로 복사해서 붙여넣으면 됩니다. 터미널(`$`)에 붙여넣으면 에러가 납니다. 클로드 대화를 끝내고 터미널로 돌아가려면 대화창에 `/exit` .

각 단계는 같은 순서로 되어 있습니다

표시	내용
💡 **왜 하나요?**	이 단계를 하는 이유
📖 **용어**	처음 나오는 말 풀이
⌨️ / 💬	입력할 것
👀 **확인할 것**	화면에서 무엇이 보이면 성공인지

시간표 한눈에 (09:00–18:00)

시간	블록	하는 것	쉬운 말
09:00–09:20	0. 오리엔테이션	앱 받기, 실행	오늘의 환자
09:20–10:00	1. 심부름꾼 복습	역할 카드, @, Ctrl+B	팀 전에 심부름꾼
10:00–10:30	2. 팀 켜기 + 첫 팀	예약 기능 기획 팀	첫 회의
10:30–10:40	☕ 휴식		
10:40–11:30	3. 🔍 리뷰 팀	혼자 vs 팀 리뷰 대결	버그 사냥
11:30–12:10	4. 💬 토론 + 직접 대화	수리 우선순위 정하기	반박 회의
12:10–12:30	🧩 오전 퀴즈		"혼자? 심부름꾼? 팀?"
12:30–13:30	🍚 점심		
13:30–14:30	5. 🛠️ 구현 팀	수리 + 예약 기능	파일 주인 정하기
14:30–15:10	6. 🚧 검문소	TaskCompleted hook	테스트 못 넘으면 완료 금지
15:10–15:20	☕ 휴식		

시간	블록	하는 것	쉬운 말
15:20–15:50	7. 🕵️ 디버깅 토론	짝꿍이 심은 버그 찾기	경쟁 가설
15:50–16:00	☕ 휴식		
16:00–17:30	8. 🚀 나만의 팀	학생이 고른 상황	2바퀴: 내가 팀장
17:30–18:00	9. 발표 & 회고		

수업 450분 + 휴식 30분 = 480분 = 8시간 (점심 별도)

> ⏱️ **"팀원이 알아서 하는데 8시간?"** — 팀 작업 한 번에 10~20분씩 걸립니다. 기다리는 동안 **팀원이 뭘 하는지 들여다보고, 멈추고, 방향을 트는 연습**이 수업의 절반입니다.

수업 전 준비물

학생 (수업 전날까지)

준비물	확인 방법 (⌨️ 터미널)	왜 필요?
Claude Code 최신 버전	`claude update` → `claude --version`	팀 기능은 실험 기능이라 최신 버전에서 가장 잘 동작
Node.js 20 이상	`node --version`	연습용 앱을 실행하고 테스트하는 프로그램 (설치할 패키지는 없음)
Git	`git --version`	앱 받기, 세이브(커밋). Windows 는 Git Bash 포함

⚠️ **Claude 사용량**: 팀원 한 명 = 클로드 한 명이라 **팀원 수만큼** 사용량이 늘어납니다. Pro 플랜은 오후에 한도에 걸릴 수 있어서 오늘은 **팀원 3명 이하, 팀원 모델은 Sonnet** 을 원칙으로 합니다. 한도에 걸리면 짝꿍 화면으로 함께 진행합니다.

강사

- 화면에 교재 **"용어 번역표"**, **"혼자·심부름꾼·팀"** 표 띄워두기
- **부록 C 정답지**를 미리 읽어두기 (블록 3 끝에 공개)
- 학생에게는 **library-app 저장소 주소만** 안내 (강의 PDF·정답지는 Harness 저장소에 있으니 그 주소는 공유하지 않기)

🎯 지루함 방지 규칙 (강사용)

1. **설명 10분 넘기지 않기.** 팀이 일하는 동안 **현황판을 같이 들여다보기**
2. 블록마다 **체크포인트**. 통과한 사람은 손 들기 / 채팅에 ✅
3. **대결 구도**: 혼자 리뷰 vs 팀 리뷰 점수판, 디버깅 토론 시간 재기
4. 빨리 끝난 학생은 ⭐ **보너스**, 늦은 학생은 🆘 **따라잡기**
5. 팀이 일하는 동안 **"지금 팀장이 뭘 기다리고 있어요?"** 를 물어보기

📢 오늘의 규칙 (학생에게 첫 화면으로)

- 팀을 시작하기 전엔 **git commit** (망쳐도 돌아갈 세이브 포인트)
- 팀원은 **3명 이하**, 팀원 모델은 **Sonnet** (사용량 절약)
- 끝난 팀원은 **퇴근시키기** (살아 있는 팀원은 계속 사용량을 씀)
- 팀이 일하는 동안 **↑↓ Enter 로 들여다보기**

블록 0

오리엔테이션 — 오늘의 환자 받기 (20분)

목표

오늘 고칠 **연습용 앱**을 내 컴퓨터로 받아서 실행해보기

0-1. 앱 받기 (8분)

💡 **왜 하나요?** 오늘 클로드 팀이 리뷰하고 고칠 **실습 대상**입니다. 일부러 **문제를 12개 심어둔** 작은 도서 대출 웹앱이라, 팀이 무엇을 찾아내는지 정답과 비교할 수 있습니다.

📖 **용어**

말	뜻
`git clone`	GitHub 에 있는 저장소를 **통째로 내 컴퓨터에 복사**
저장소 주소	`https://github.com/아이디/저장소이름.git` 모양. clone 에는 **이 주소만** 넣을 수 있음

⌨️ **터미널**

```
mkdir -p ~/projects && cd ~/projects                    # 홈(~) 아래 projects 폴더를 만들고(-p = 이미 있으면 그냥 넘어감) 그 폴더로 이동. && = 앞 명령이 성공하면 뒤 명령 실행
git clone https://github.com/NujabesCode/library-app.git   # 연습용 앱 저장소를 내 컴퓨터로 복사 → library-app 폴더가 생김
cd library-app                                          # 받은 앱 폴더로 들어가기
git log --oneline                                       # 첫 커밋 1개가 보이면 정상 (오늘 실습 기록은 이 위에 쌓임)
```

⚠️ GitHub 화면의 **폴더 주소**(`.../tree/main/...`)를 넣으면 `repository not found`, 주소 뒤에 단어를 두 개 이상 붙이면 `Too many arguments` 에러가 납니다. **위 명령 그대로** 복사하세요.

👀 **확인할 것:** `git log --oneline` 에 `init: 팀 에이전트 실습용 도서 대출 앱` 한 줄이 보이면 성공

0-2. 실행해보기 (7분)

💡 **왜 하나요?** 리뷰 전에 **앱이 원래 어떻게 동작하는지** 직접 눌러봐야, 나중에 팀이 "고쳤다" 고 할 때 무엇이 달라졌는지 알 수 있습니다.

📖 **용어**

말	뜻
`npm test`	`package.json` 에 적힌 **테스트 실행 명령**(`node --test`)을 돌림 → `test/` 폴더의 테스트 실행
`npm start`	`package.json` 에 적힌 **시작 명령**(`node server.js`)을 돌림 → 웹 서버가 켜짐
`localhost:3000`	**내 컴퓨터**(localhost)의 3000번 문(포트)에서 켜진 웹사이트
Ctrl + C	터미널에서 실행 중인 프로그램(서버)을 **끄기**

⌨️ **터미널**

```
npm test    # 테스트 실행 → "pass 2" 가 보이면 정상
npm start   # 서버 시작 → 브라우저에서 http://localhost:3000 접속. 끌 때는 Ctrl + C
```

브라우저에서: **가입 → 로그인 → 검색 → 대출 → 반납** 눌러보기 → 터미널로 돌아와 **Ctrl + C** 로 서버 끄기

👀 **확인할 것**: 테스트 `pass 2`, 브라우저에 책 5권 목록

⚠️ 이 앱에는 **일부러 심어둔 문제가 12개** 있습니다. 지금 찾지 말고, 오늘 클로드 팀이 찾게 합니다.

0-3. 오늘의 목표 선언 (5분)

> "오늘은 클로드를 **한 명이 아니라 팀으로** 부립니다. 그리고 **언제 팀을 쓰면 안 되는지**도 배웁니다. 팀은 비쌉니다."

교재의 **혼자 · 심부름꾼 · 팀** 표를 같이 읽기

✅ 체크포인트

- [ ] `npm test` 가 2개 통과
- [ ] 브라우저에서 책 목록이 보였다
- [ ] `git log --oneline` 에 첫 커밋이 있다

🆘 따라잡기: clone 이 안 되면 → github.com/NujabesCode/library-app 에서 **Code → Download ZIP** → `~/projects` 에 압축 풀기 → 그 폴더에서 ⌨️ `git init && git add . && git commit -m "init"`

블록 1

심부름꾼(서브에이전트) 복습 (40분)

목표

팀을 쓰기 전에, 더 가볍고 싼 **심부름꾼** 으로 할 수 있는 걸 먼저 써보기

> 💡 **이 블록을 먼저 하는 이유**: 팀 에이전트는 비쌉니다. 많은 일은 **심부름꾼 한 명**으로 충분합니다. 심부름꾼이 뭔지 알아야 "이건 팀이 필요한 일인가?" 를 판단할 수 있습니다.

📖 용어

말	뜻
서브에이전트 (Subagent)	**심부름꾼.** 메인 클로드가 곁가지 일을 다른 클로드에게 보내고 **결과 요약만** 돌려받는 방식
메인 클로드	내가 대화창에서 대화하고 있는 클로드
컨텍스트 (머릿속)	클로드가 한 번에 기억하는 대화·파일 내용. 너무 많이 쌓이면 헷갈리고 비싸짐

1-1. 기본 심부름꾼 써보기 (8분)

💡 **왜 하나요?** Claude Code 에는 **이미 만들어진 심부름꾼**이 있습니다. 그중 **Explore** 는 파일을 **읽기만** 하는 탐색 전문가입니다. 파일 수십 개를 메인 클로드가 직접 열면 그 내용이 전부 대화에 쌓이지만, Explore 에게 시키면 **탐색은 심부름꾼 머릿속에서** 하고 **깔끔한 요약만** 돌아옵니다. 이 "보내고 → 요약만 받는" 느낌을 처음 체험하는 단계입니다.

📖 용어

말	뜻
Explore	Claude Code 에 **기본으로 들어 있는** 읽기 전용 탐색 심부름꾼
프로젝트 구조	어떤 파일·폴더가 있고 각자 무슨 일을 하는지

⌨️ 터미널

```
claude   # 이 폴더에서 Claude Code 대화 시작 → 화면이 클로드 대화창으로 바뀜
```

💬 클로드 대화창

```
Explore 서브에이전트로 이 프로젝트 구조를 파악해서, 파일마다 하는 일을 한 줄씩 표로 정리해줘.
```

👀 **확인할 것** (버전마다 모양은 조금씩 다름)

```
● Explore(프로젝트 구조 파악)              ← ① 메인 클로드가 심부름꾼을 보냈다는 표시
  ⎿  Read(server.js)                     ← ② 심부름꾼이 파일을 읽는 중
     +5 more tool uses                   ← 나머지는 접혀 있음
  ⎿  Done (8 tool uses · 18k tokens · 25s)  ← ③ 끝. 도구 8번 사용, 25초
● ┆ 파일 ┆ 하는 일 ┆ ...                  ← ④ 메인 클로드는 요약 표만 받음
```

보는 곳	뜻
`● Explore(...)` 줄	메인이 직접 안 하고 **심부름꾼을 보냈다**
`Read(...)` 가 지나가다 접힘	파일 읽기는 **심부름꾼 머릿속**에서만. 메인 대화엔 안 쌓임
`Done (...)` 다음에 표만	**요약만** 돌아왔다

> 💡 **Ctrl + O** 로 접힌 내용을 펼쳐서 심부름꾼이 한 일을 자세히 볼 수 있습니다 (다시 누르면 접힘)

1-2. 역할 카드 만들기 (15분)

💡 **왜 하나요?** Explore 같은 기본 심부름꾼 말고, **내가 원하는 역할의 심부름꾼**을 만들 수 있습니다. 한 번 파일로 만들어두면 매번 "너는 테스트 담당이고, 코드는 고치지 말고..." 라고 설명할 필요가 없습니다. 이 **역할 카드**는 블록 3 에서 **팀원으로도 그대로 재사용**합니다.

📖 용어

말	뜻
`.claude/agents/`	이 프로젝트용 역할 카드를 두는 폴더. 파일 하나 = 심부름꾼 하나
역할 카드 파일	위쪽 `---` 사이에 **설정**, 아래에 **업무 지시** 를 적은 `.md` 파일
frontmatter	파일 맨 위 `---` 와 `---` 사이의 **설정 부분**
`description`	**언제 이 심부름꾼을 부를지** 설명. 클로드가 이걸 읽고 알아서 부름
`tools`	이 심부름꾼이 **쓸 수 있는 도구 목록**. 적힌 것만 사용 가능
`model`	이 심부름꾼이 쓸 **두뇌(모델)**
본문	`---` 아래 내용. 심부름꾼이 받는 **업무 지시서**

📖 **도구 (tools) 풀이** — 클로드가 일할 때 쓰는 손발입니다

도구	하는 일	비유
Read	파일 **내용을 읽기**	책 펼쳐 읽기
Grep	여러 파일 **안에서 글자 찾기** (예: `borrowBook` 이 어디 쓰였나)	Ctrl + F 를 폴더 전체에
Glob	**파일 이름 모양으로** 파일 찾기 (예: `test/*.js` → test 폴더의 js 파일 전부)	탐색기에서 `*.js` 로 검색
Bash	**터미널 명령 실행** (예: `npm test`)	터미널에 직접 타이핑
Edit / Write	파일 **고치기 / 새로 쓰기**	펜으로 고쳐 쓰기

> 💡 **일부러 Edit 와 Write 를 빼는 이유**: "코드는 고치지 마" 라고 **말로만** 하면 클로드가 가끔 고칩니다. 도구 목록에서 **아예 빼버리면** 고치고 싶어도 못 고칩니다. 말이 아니라 **구조로 막는** 것입니다.

📖 **모델 (model) 풀이**

값	특징
`sonnet`	빠르고 비용이 적당. **오늘 기본값**
`opus`	가장 똑똑하지만 비쌈. 어려운 판단용
`haiku`	가장 빠르고 쌈. 단순 작업용
`inherit`	메인 클로드와 같은 모델

💬 **클로드 대화창**

```
.claude/agents/test-runner.md 서브에이전트를 만들어줘.
- description: 테스트를 실행하고 실패 원인을 요약할 때 사용
- tools: Read, Grep, Glob, Bash
- model: sonnet
- 본문: npm test 를 실행하고, 실패하면 원인과 파일:줄만 보고해. 코드는 절대 고치지 마.
```

👀 **확인할 것**: 만들어진 파일을 열어서 (VS Code 에서 `.claude/agents/test-runner.md`) 아래 모양인지 같이 읽기

```
---
name: test-runner
description: 테스트를 실행하고 실패 원인을 요약할 때 사용
tools: Read, Grep, Glob, Bash
model: sonnet
---

npm test 를 실행하고, 실패하면 원인과 파일:줄만 보고해. 코드는 절대 고치지 마.
```

1-3. 지목해서 부르기 (7분)

💡 **왜 하나요?** 그냥 "테스트 돌려줘" 라고 하면 클로드가 **직접** 할 수도, 심부름꾼을 **보낼** 수도 있습니다. `@` 로 지목하면 **반드시 그 심부름꾼이** 일합니다.

📖 **용어**

말	뜻
`@` **멘션**	대화창에서 `@` 를 치면 파일·심부름꾼 목록이 뜸 → 골라서 **콕 집어 부르기**

💬 **클로드 대화창**: `@` 를 입력 → 목록에서 `test-runner (agent)` 선택 → 이어서 입력

```
@"test-runner (agent)" 테스트 돌려줘
```

👀 **확인할 것**: `● test-runner(...)` 줄이 생기고, 결과로 "2개 통과" 같은 요약만 돌아옴

1-4. 백그라운드로 보내기 (10분)

💡 **왜 하나요?** 심부름꾼이 일하는 동안 기다리기만 하면 시간이 아깝습니다. **백그라운드**로 보내면 심부름꾼은 뒤에서 일하고, 나는 그동안 **다른 질문**을 할 수 있습니다. 팀 에이전트의 "여럿이 동시에 일한다" 감각의 맛보기입니다.

📖 **용어**

말	뜻
포그라운드	심부름꾼이 끝날 때까지 **대화가 멈춰서 기다림** (기본)
백그라운드	심부름꾼은 **뒤에서** 일하고, 나는 계속 대화 가능
Ctrl + B	지금 실행 중인 일을 **백그라운드로 보내기**

💬 **클로드 대화창**

```
test-runner 서브에이전트로 테스트를 돌리고, src 폴더에 테스트가 없는 함수 목록도 찾아줘.
```

→ 심부름꾼이 일하기 시작하면 바로 **Ctrl + B** → 입력창이 다시 열리면:

```
README 를 읽고 API 목록을 한 줄로 요약해줘.
```

👀 **확인할 것**: README 요약이 먼저 오고, **심부름꾼 결과가 나중에 따로 도착**

💬 **강사 질문**: "test-runner 는 결과만 돌려줬어요. **다른 심부름꾼과 상의**했나요?" → 아니요. 심부름꾼은 **보고만** 합니다. 서로 상의하고 반박하는 건 **팀**만 할 수 있습니다 (교재 2단계 표)

✅ 체크포인트

- [ ] Explore 가 일하는 `● Explore(...)` 줄을 봤다
- [ ] `.claude/agents/test-runner.md` 가 있고, tools 에 Edit 가 **없는 이유**를 말할 수 있다
- [ ] `@` 로 지목해서 불렀다
- [ ] Ctrl + B 로 백그라운드에 보냈다

⭐ 보너스: `~/.claude/agents/` (내 **모든** 프로젝트용 폴더)에 심부름꾼 하나 만들기 (예: 코드를 5살에게 설명하듯 풀어주는 `explain-like-5`)

🆘 따라잡기: 1-2 를 클로드에게 부탁하는 것만 성공해도 OK

블록 2

팀 켜기 + 첫 팀 (30분)

목표

팀 기능을 켜고, 팀원이 일하는 모습을 **현황판**으로 들여다보는 연습

📖 **용어**

말	뜻
팀 에이전트 (Agent Teams)	클로드 여러 명이 **한 팀**으로 일하는 기능. 서로 메시지로 대화함
팀장 (Lead)	내가 대화하는 원래 클로드. 팀원을 뽑고 일을 나누고 결과를 합침
팀원 (Teammate)	팀장이 뽑은 **독립된 클로드**. 각자 자기 머릿속에서 일함
현황판 (Agent panel)	입력창 **아래** 팀원 목록. 누가 일하는 중인지 보임

2-1. 팀 기능 켜기 (7분)

💡 **왜 하나요?** 팀 에이전트는 아직 **실험 기능**이라 **기본으로 꺼져** 있습니다. 설치할 건 없고, **설정 파일에 한 줄**만 넣으면 켜집니다. **이 프로젝트에서만** 켜는 이유는, 전체 설정에 켜두면 다른 프로젝트에서도 클로드가 마음대로 팀을 꾸려 사용량이 새기 때문입니다.

📖 용어

말	뜻
`.claude/settings.json`	**이 프로젝트용** Claude Code 설정 파일
`env`	환경 변수 (프로그램에게 주는 스위치) 를 적는 칸
`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS`	팀 기능 스위치. `"1"` = 켜기, `"0"` = 끄기
`/exit`	클로드 대화를 끝내고 터미널로 돌아가기

💬 클로드 대화창

```
이 프로젝트의 .claude/settings.json 에 env 로 CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1 을 추가해줘.
기존 설정이 있으면 합쳐줘. 저장 후 파일 전체를 보여줘.
```

👀 **확인할 것:** 이 내용이 들어 있으면 성공

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

💬 `/exit` → ⌨️ 다시 `claude` (새 설정으로 깨끗하게 시작)

2-2. 첫 팀: 예약 기능 기획 회의 (15분)

💡 **왜 하나요?** 첫 팀은 **코드를 고치지 않는 일**(기획)로 시작합니다. 코드 수정은 팀원끼리 같은 파일을 건드리는 사고가 날 수 있지만, 기획은 **각자 다른 관점에서 동시에 생각**만 하면 되니 실패할 일이 적습니다. 오후에 실제로 만들 **"예약 기능"** 을 미리 설계해 둡니다.

📖 용어

말	뜻
예약 기능	재고가 0인 책을 예약해두면, 반납될 때 **순서대로** 받을 수 있는 기능
악마의 변호인 (critic)	일부러 **반대 의견**을 내서 빠뜨린 문제를 찾는 역할
`docs/`	기획·리뷰 문서를 모아둘 폴더 (클로드가 만들어 줌)

💬 클로드 대화창

```
이 도서 대출 앱에 "예약 기능"(재고가 0인 책을 예약해두면 반납될 때 순서대로 받을 수 있는 기능)을 추가하려고 해.
agent team 으로 팀원 3명을 뽑아줘. 이름은 ux, arch, critic, 모델은 모두 Sonnet.
- ux: 사용자 입장에서 화면 흐름과 안내 문구
- arch: 데이터 구조와 API 설계 (src/books.js, server.js 기준)
- critic: 악마의 변호인. 이 기능의 문제점과 빠뜨린 경우를 찾기
코드는 고치지 말고, 결과를 합쳐서 docs/reservation-plan.md 에 써줘.
```

> 💡 **"agent team 으로" 를 꼭 넣는 이유**: 안 넣으면 클로드가 팀 대신 **심부름꾼**을 쓸 때가 있습니다. 둘은 현황판에 같이 보여서 헷갈립니다.

2-3. 현황판 조작 연습 (8분) — 팀이 일하는 동안

💡 **왜 하나요?** 팀장만 보고 있으면 팀원이 **엉뚱한 방향으로 오래 달리는 걸** 놓칩니다. 현황판에서 팀원 화면을 들여다보고 직접 말을 거는 게 팀 운영의 기본입니다.

해보기	키	설명
팀원 고르기	**↑ / ↓**	입력창 **아래** 현황판에서
그 팀원 화면 보기	**Enter**	그 팀원이 지금까지 한 일 + **직접 말 걸기** 가능
팀장 화면으로 돌아오기	**Esc**	⚠️ 팀원 화면에서 Esc 는 **그 팀원 작업을 멈춤** → 실험해보고 "계속해" 라고 말해주기
할 일 보드	**Ctrl + T**	팀 전체의 할 일 목록 (대기 / 진행 중 / 완료)

→ critic 화면에 들어가서 (↑↓ 로 고르고 Enter) 💬 직접 말 걸기:

```
초등학생이 쓴다고 생각하고 문제점을 하나 더 찾아봐.
```

끝나면 💬 팀장 화면에서:

```
팀원 모두 퇴근시켜줘.
```

> 💡 **퇴근시키는 이유**: 팀원은 일이 끝나도 살아 있는 동안 계속 사용량을 씁니다.

👀 **확인할 것**: 현황판에 `ux`, `arch`, `critic` 3명 / `docs/reservation-plan.md` 파일 생성

⚠️ 팀원 줄이 갑자기 사라져도 **죽은 게 아닙니다.** 모두가 쉬면 30초 뒤 **숨겨질** 뿐입니다.

✅ 체크포인트

- [ ] 현황판에 팀원 3명이 보였다 (심부름꾼이 아니라 **팀원**)
- [ ] 팀원 화면에 들어가 직접 말을 걸었다
- [ ] `docs/reservation-plan.md` 가 생겼다

🆘 따라잡기: 팀 대신 심부름꾼이 생기면 → 💬 `서브에이전트 말고 agent team 으로 다시 해줘`

⭐ 보너스: mac + tmux 사용자는 ⌨️ `claude --teammate-mode auto` 로 **분할 화면** 체험 (Windows·VS Code 터미널은 미지원)

☕ 휴식 (10분)

💡 쉬기 전에 지금까지 만든 걸 **세이브**해 둡니다. 오후에 팀이 코드를 고치다 망쳐도 여기로 돌아올 수 있습니다.

⌨️ **터미널** (클로드 대화 중이면 새 터미널을 하나 더 열어서)

```
git add . && git commit -m "docs: 예약 기능 기획"   # 지금까지 만든 파일(역할 카드, 설정, 기획 문서)을 담아서(add) 세이브(commit)
```

블록 3

🔍 리뷰 팀 — 혼자 vs 팀 대결 (50분)

목표

같은 코드를 **혼자 리뷰** 한 것과 **팀으로 리뷰** 한 것을 비교해서, 팀이 **언제 이득인지** 숫자로 확인

> 💡 **이 블록이 오늘의 핵심인 이유**: "팀이 좋다" 는 말만 들으면 모든 일에 팀을 씁니다. 같은 일을 두 방식으로 해보고 **찾은 개수**와 **쓴 사용량**을 나란히 놓아야 판단 기준이 생깁니다.

📖 **용어**

말	뜻
코드 리뷰	코드를 읽고 **문제점을 찾아 보고**하는 일 (고치지는 않음)
심각도	문제가 얼마나 급한지: 높음 / 중간 / 낮음
재현	문제가 **진짜로 일어나는지** 실제로 실행해서 보여주기

3-1. 🧑 혼자 리뷰 (10분) — 기준점

💡 **왜 하나요?** 팀과 비교할 **기준 점수**를 만듭니다. **새 터미널**에서 하는 이유는 앞에서 나눈 대화(기획 회의 내용)가 섞이지 않은 **깨끗한 클로드**로 리뷰해야 공정하기 때문입니다.

⌨️ **새 터미널**을 하나 더 열고

```
cd ~/projects/library-app && claude   # 앱 폴더로 이동한 뒤 새 클로드 대화 시작
```

💬 **클로드 대화창**

```
이 앱의 문제점을 리뷰해줘. 보안, 성능, 버그, 테스트 부족 무엇이든.
서브에이전트나 팀은 쓰지 말고 너 혼자 해줘. 코드는 고치지 말고,
문제마다 한 줄씩 번호를 매겨서 docs/review-solo.md 에 써줘.
```

👀 **확인할 것**: 몇 개 찾았는지 **숫자를 칠판에** 적기 → 💬 `/exit` 로 이 터미널은 닫기

3-2. 역할 카드 3장 (10분)

💡 **왜 하나요?** 리뷰를 **보안·성능·테스트** 세 관점으로 나눕니다. 클로드 한 명은 한 가지 관점에 꽂히는 버릇이 있어서, 관점별로 **전문가를 따로** 두면 빠뜨리는 게 줄어듭니다. 블록 1 에서 배운 역할 카드로 만들면 팀원에게 **그대로 재사용**할 수 있습니다.

📖 **용어**

말	뜻
security-reviewer	보안 리뷰어: 비밀번호, 로그인, 해킹 가능성
perf-reviewer	성능 리뷰어: 느린 곳, 서버가 죽을 수 있는 곳
test-reviewer	테스트 리뷰어: 로직 버그, 빠진 테스트
"Bash 는 재현용으로만"	터미널 명령은 **문제를 증명할 때만** 쓰고 파일은 고치지 말라는 뜻

💬 **원래 터미널의 클로드 대화창** (팀 기능이 켜진 쪽)

```
.claude/agents/ 에 팀원용 역할 카드 3개를 만들어줘: security-reviewer, perf-reviewer, test-reviewer.
- 모두 tools 는 Read, Glob, Grep, Bash (Bash 는 재현용으로만, 파일 수정 금지)
- model: sonnet
- 결과는 표로: 번호 ¦ 심각도(높음/중간/낮음) ¦ 파일:줄 ¦ 문제 ¦ 재현 방법 ¦ 고치는 방법
- 추측이면 "추측" 이라고 표시
각 카드의 description 은 그 역할이 언제 쓰이는지 분명하게.
```

👀 **확인할 것:** `.claude/agents/` 에 파일 3개 + 블록 1 의 `test-runner.md`

3-3. 🏢 팀 리뷰 (20분)

💡 **왜 하나요?** 세 전문가가 **동시에** 각자 맡은 파일을 봅니다. **파일 범위를 나눠주는 이유**는 같은 곳을 셋이 중복해서 보면 사용량만 세 배가 되기 때문입니다.

📖 **용어**

말	뜻
에이전트 타입	팀원을 뽑을 때 쓸 **역할 카드 이름** (3-2 에서 만든 것)

💬 **클로드 대화창**

```
agent team 으로 리뷰 팀을 꾸려줘. 팀원 3명:
- sec: security-reviewer 에이전트 타입. src/auth.js, server.js, public/index.html 의 보안
- perf: perf-reviewer 에이전트 타입. src/books.js, server.js 의 성능·안정성(서버가 죽을 수 있는 경우 포함)
- test: test-reviewer 에이전트 타입. src/books.js 의 로직 버그와 test/ 의 빠진 테스트
코드는 고치지 마. 끝나면 결과를 합쳐서 docs/review-team.md 에 써줘. 같은 문제는 하나로 합치고 심각도 순으로 정렬해.
```

팀이 일하는 동안:

- **Ctrl + T** 로 할 일 보드 보기 (누가 무엇을 하는 중인지)
- `sec` 화면에 들어가서 (↑↓ → Enter) 💬 `재현해본 것만 '확인됨' 으로 표시해줘` 라고 **직접** 지시해보기

👀 **확인할 것**: `docs/review-team.md` 에 심각도 순 표

3-4. 🏆 점수판 (10분)

💡 **왜 하나요?** 느낌이 아니라 **숫자**로 비교합니다.

1. `docs/review-solo.md` vs `docs/review-team.md` 찾은 개수 비교
2. 강사가 **부록 C 정답지** 공개 → 각자 몇 개 맞혔는지 채점
3. 💬 `/usage` 로 지금까지 쓴 사용량 확인 (📖 `/usage` = 사용량 보기 명령)
4. 칠판에 "혼자 평균 / 팀 평균 / 사용량" 기록

💬 **강사 질문**: "팀이 더 많이 찾았나요? 그럼 **사용량**은 몇 배였나요? 이 정도 차이면 매번 팀을 쓸 건가요?"

✅ 체크포인트

- [ ] 역할 카드 3장이 `.claude/agents/` 에 있다
- [ ] `review-solo.md` 와 `review-team.md` 가 둘 다 있다
- [ ] 정답지와 비교해 점수를 냈다

🆘 따라잡기: 3-2 를 건너뛰고, 3-3 프롬프트에서 "에이전트 타입" 부분을 빼고 실행 (역할을 지시에 직접 적기)

블록 4

💬 토론 + 직접 대화 (40분)

목표

팀원끼리 **서로 반박**하게 해서 리뷰 결과를 다듬고, **고칠 순서**를 정하기

> 💡 **이 블록을 하는 이유**: 심부름꾼은 보고만 하고 끝나지만, 팀원은 **서로의 결과를 보고 반박**할 수 있습니다. 리뷰에는 **거짓 경보**(실제로는 문제가 아닌데 문제라고 한 것)가 섞이기 마련이라, 반박으로 걸러내면 결과가 정확해집니다. 이게 팀을 쓰는 가장 큰 이유입니다.

📖 용어

말	뜻
반박	다른 팀원의 결론이 틀렸다고 **근거를 들어** 말하기
철회	반박을 받고 증명하지 못하면 그 주장을 **취소**
메신저 (Mailbox)	팀원끼리 **이름으로** 메시지를 주고받는 기능. 자동으로 배달됨

4-1. 반박 회의 (15분)

리뷰 팀이 아직 살아 있으면 그대로, 퇴근했다면 다시 뽑아서:

💬 **클로드 대화창**

```
리뷰 팀원 3명에게 전달해줘: docs/review-team.md 를 같이 읽고,
다른 팀원이 찾은 문제 중 과장됐거나 틀렸다고 생각하는 것을 그 팀원에게 직접 반박해.
반박당한 팀원은 재현으로 증명하거나 철회해. 합의된 결과로 review-team.md 를 고쳐줘.
```

👀 **확인할 것**: 현황판에서 팀원끼리 메시지가 오가는 것 (누가 누구에게 보냈는지)

4-2. 내가 직접 끼어들기 (10분)

💡 **왜 하나요?** 모든 걸 팀장을 거쳐 전달하면 느리고 말이 바뀝니다. 팀원에게 **직접** 물어볼 수 있습니다.

`perf` 팀원 화면으로 (↑↓ → **Enter**) 💬

```
"서버가 죽는 문제"가 있다면 그게 제일 급해. 재현 명령어를 정확히 알려줘.
```

👀 **확인할 것**: 팀장을 거치지 않고 perf 가 바로 답함

4-3. 수리 계획 (10분)

💡 **왜 하나요?** 오후에 팀원 3명이 **나눠서 고칠** 겁니다. 문제마다 **어느 파일을 고쳐야 하는지**, **누가 맡을지**를 미리 정해야 두 팀원이 같은 파일을 건드리는 사고를 막을 수 있습니다.

📖 **용어**

말	뜻
담당자 열	표에서 "누가 고칠지" 칸. books / auth / web / 팀장

💬 **클로드 대화창** (팀장 화면에서)

```
review-team.md 를 바탕으로 docs/fix-plan.md 를 만들어줘.
- 심각도 높음부터
- 각 문제를 고칠 파일 (src/books.js / src/auth.js / public/index.html / server.js) 열 추가
- 오후에 팀원 3명(books, auth, web)이 나눠 고칠 거니까 담당자 열도 추가. server.js 는 팀장 담당
```

4-4. 퇴근 + 저장 (5분)

💬 `팀원 모두 퇴근시켜줘.`

⌨️ **터미널**

```
git add . && git commit -m "docs: 리뷰 결과와 수리 계획"   # 리뷰 문서와 수리 계획을 세이브
```

✅ 체크포인트

- [ ] 팀원끼리 반박 메시지가 오가는 걸 봤다
- [ ] 팀원에게 직접 질문하고 답을 받았다
- [ ] `docs/fix-plan.md` 에 담당자 열이 있다

⭐ 보너스: 반박으로 **철회된 문제**가 있었는지 찾아보기 (팀 토론이 거짓 경보를 걸러냈는가?)

🧩 오전 복습 퀴즈 (20분)

퀴즈 1. "혼자? 심부름꾼? 팀?" (10분, 팀 대항)

상황	정답	이유
오타 하나 고치기	🧑 혼자	너무 작음
로그 5만 줄에서 에러 원인 찾기	🏃 심부름꾼	결과만 필요, 긴 로그가 메인 대화에 안 쌓이게
PR 하나를 보안·성능·테스트로 동시에 리뷰	🏢 팀	관점이 여러 개
A 끝나야 B, B 끝나야 C 인 작업	🧑 혼자	순서가 있으면 팀원이 서로 기다리기만 함
원인 모르는 버그, 가설이 여러 개	🏢 팀	서로 반박하는 토론이 필요
같은 파일 하나를 크게 고치기	🧑 혼자	여럿이 같은 파일 = 덮어쓰기 사고
프론트·백엔드·테스트를 다른 파일에서 개발	🏢 팀	파일이 안 겹치게 나눌 수 있음
테스트만 돌리고 실패 요약	🏃 심부름꾼	상의할 필요 없음

퀴즈 2. "왜 이렇게 됐지?" (10분)

증상	원인
팀원이 내가 아까 설명한 배경을 모름	팀원은 **팀장과 나눈 대화를 모름** → 지시에 직접 적기
팀원 줄이 현황판에서 사라짐	**30초 뒤 숨김** (죽은 게 아님)
팀 대신 심부름꾼이 생김	"**agent team 으로**" 를 안 적음
두 팀원이 고친 내용 중 하나가 사라짐	**같은 파일**을 둘이 고침 → 파일 주인 정하기
팀원 화면에서 `/model` 이 안 먹힘	팀원 모델은 **뽑을 때 고정**
사용량이 갑자기 확 줄었음	팀원 수만큼 사용 → 끝난 팀원 퇴근

🍚 점심 (60분)

> 점심 전 한마디: "오후엔 팀이 **코드를 고칩니다.** 같은 파일을 둘이 고치면 사고가 납니다. 점심 먹으면서 `fix-plan.md` 의 담당자 열을 한 번 더 보세요."

블록 5

🛠️ 구현 팀 — 수리 + 예약 기능 (60분)

목표

파일 주인을 정하고, **계획을 먼저 확인**한 뒤, 팀이 동시에 고치고 기능까지 추가

> 💡 **오전과 다른 점**: 오전엔 읽기만 했지만 이번엔 팀원들이 **동시에 파일을 고칩니다.** 그래서 ① 망쳐도 돌아갈 세이브 ② 파일 주인 ③ 내가 먼저 확인하는 계획, 세 가지 안전장치를 먼저 겁니다.

5-1. 안전장치 (10분)

💡 왜 하나요?

- `git status` : 커밋 안 한 변경이 있으면 팀이 망쳤을 때 **어디로 돌아갈지** 모릅니다
- **허락 목록**: 팀원 3명이 테스트를 돌릴 때마다 "실행해도 돼요?" 를 물으면 **허락 요청 폭탄**이 쏟아집니다. 안전한 명령은 미리 허락해 둡니다

📖 용어

말	뜻
`permissions.allow`	설정 파일의 **"묻지 말고 해도 되는 명령" 목록**
`Bash(npm test:*)`	`npm test` 로 **시작하는** 터미널 명령은 허락 없이 실행

⌨️ 터미널

```
git status   # "nothing to commit, working tree clean" 이 나와야 함. 아니면 git add . && git commit 먼저
```

💬 클로드 대화창

```
.claude/settings.json 의 permissions.allow 에 "Bash(npm test:*)", "Bash(node --test:*)" 를 추가해줘. 기존 설정은 유지해줘.
```

5-2. 파일 소유표 + 할 일 보드 (10분)

💡 왜 하나요?

- **파일 소유표**: 팀원은 모두 **같은 폴더**에서 일합니다. 두 명이 같은 파일을 고치면 **서로 덮어씁니다.** 파일마다 주인을 정하면 막을 수 있습니다
- **계획 모드**: 팀원을 뽑기 **전에** 계획부터 보고 확인합니다. 바로 뽑으면 팀원들이 이미 코드를 고치기 시작한 뒤라 되돌리기 어렵습니다
- **팀원은 대화를 모름**: 팀원은 팀장과 내가 나눈 대화를 모릅니다. 그래서 각 팀원에게 줄 **업무 지시서**를 미리 보고 빠진 게 없는지 확인합니다

📖 용어

말	뜻
계획 모드 (plan mode)	클로드가 **읽기만 하고 계획을 세우는** 모드. 파일을 고치지 않음
Shift + Tab	대화창에서 모드를 차례로 바꾸는 키. 화면 아래에 `plan mode` 가 보이면 멈춤
할 일 보드	팀 전체의 할 일 목록. 대기 / 진행 중 / 완료
선행 작업	"A 가 끝나야 B 를 시작할 수 있다". 끝나기 전엔 B 가 잠김
업무 지시서	팀원에게 주는 지시: 역할 / 범위 / 배경 / 할 일 / 결과물

💬 **클로드 대화창**: **Shift + Tab** 을 눌러 `plan mode` 로 바꾼 뒤

```
docs/fix-plan.md 와 docs/reservation-plan.md 를 바탕으로 구현 팀 계획을 세워줘.
팀원 3명(books, auth, web), 모델은 Sonnet. 파일 소유표:
- books: src/books.js, test/books.test.js
- auth: src/auth.js, test/auth.test.js (새로 만들기)
- web: public/index.html
- 팀장: server.js (API 라우트 추가·수정은 팀장만)
할 일 보드는 팀원 1명당 4~6개, 선행 작업 관계도 넣어줘 (예: 예약 API 는 books 의 예약 로직이 끝난 뒤).
각 팀원 업무 지시서(역할/범위/배경/할 일/결과물)를 먼저 보여주고, 내가 OK 하면 그때 팀원을 뽑아.
```

👀 **확인할 것**: 지시서를 읽으며 **파일 범위가 서로 겹치지 않는지** 확인 → 괜찮으면 다음 단계

> 💡 **server.js 를 팀장만 고치는 이유**: 서버 주소(라우트)는 books·auth·web 모두가 필요로 하는 **공통 파일**입니다. 여럿이 손대면 충돌이 나니 한 명(팀장)이 마지막에 합칩니다.

5-3. 팀 가동 (30분)

💡 **왜 이렇게 지시하나요?**

- **할 일마다** `npm test` : 고칠 때마다 다른 기능을 깨뜨리지 않았는지 바로 확인
- **자기 파일만 커밋**: 남의 파일까지 `git add .` 하면 다른 팀원이 고치던 중간 상태가 섞여 들어감
- **"직접 구현하지 마"**: 팀장이 기다리지 않고 **혼자 코드를 써버리는** 경우가 있음

💬 **클로드 대화창**: **Shift + Tab** 으로 계획 모드를 끄고 (일반 모드로)

```
OK. 팀원을 뽑아서 진행해줘. 팀원들은 각자 할 일이 끝날 때마다 npm test 를 돌리고,
자기 파일만 git add 해서 "팀원이름: 한 일" 형식으로 커밋해. 너는 팀원들이 끝날 때까지 기다리고 직접 구현하지 마.
```

팀이 일하는 동안:

- **Ctrl + T**: 🔒 잠긴 일이 선행 작업이 끝나면 **풀리는** 걸 관찰
- 팀장이 직접 코드를 쓰기 시작하면 → 💬 `팀원들이 끝날 때까지 기다려. 직접 구현하지 마`
- 팀원이 오래 멈춰 있으면 → 그 팀원 화면에 들어가 💬 `지금 어디까지 했어? 막힌 게 있어?`

5-4. 확인 (10분)

⌨️ 터미널

```
git log --oneline   # 팀원별 커밋("books: ...", "auth: ...")이 보이는지 확인
npm test            # 모든 테스트 통과 확인
npm start           # 서버 켜고 브라우저에서 예약 기능 눌러보기 → 끝나면 Ctrl + C
```

💬 클로드 대화창

```
팀원 모두 퇴근시키고, 할 일 보드에서 끝나지 않은 일이 있으면 알려줘.
```

✅ 체크포인트

- [ ] 팀원 업무 지시서를 **내가 먼저 확인**했다
- [ ] `git log` 에 팀원별 커밋이 있다
- [ ] `npm test` 통과 + 브라우저에서 예약 기능이 보인다

🆘 따라잡기: 사용량이 부족하면 → 예약 기능은 빼고 **심각도 높음 문제만** 팀원 2명(books, auth)으로

⭐ 보너스: ⌨️ `curl --path-as-is http://localhost:3000/../server.js` 가 이제 **막히는지** 확인 (서버를 켠 상태에서. 📖 `curl` = 터미널에서 웹 주소에 요청 보내기. 예전엔 이걸로 서버 소스가 보였음)

블록 6

🚧 검문소 — 테스트 못 넘으면 완료 금지 (40분)

목표

hook(검문소) 으로, 팀원이 테스트를 깨뜨린 채 "완료" 하는 걸 **기계적으로** 막기

> 💡 **이 블록을 하는 이유**: 팀원은 "다 했어요!" 하고 쉬려는 경향이 있습니다. 팀원 여럿이 동시에 일하면 내가 일일이 확인할 수 없습니다. 그래서 **"테스트를 통과해야만 완료"** 라는 규칙을 사람 대신 **자동 검문소**에 맡깁니다.

📖 **용어**

말	뜻
hook (훅)	**정해진 순간에 자동으로 실행**되는 명령. 하네스 강의의 그 검문소
TaskCompleted	팀원이 할 일을 **"완료"로 표시하려는 순간** 실행되는 hook
종료 코드 (exit code)	명령이 끝날 때 남기는 숫자. 0 = 성공, 그 외 = 실패
`exit 2`	hook 에서 **2 로 끝나면 → 그 행동을 막음** (완료 거부). 1 은 못 막음
`1>&2`	명령 결과를 **에러 출력(stderr)** 으로 보내기 → 이 내용이 팀원에게 피드백으로 전달됨
`||`	앞 명령이 **실패하면** 뒤 명령 실행

📖 `npm test 1>&2 ¦¦ exit 2` 를 한 문장으로: **"테스트를 돌려서 결과를 팀원에게 보여주고, 실패하면 완료를 막아라"**

6-1. 검문소 설치 (10분)

💬 **클로드 대화창**

```
.claude/settings.json 에 TaskCompleted hook 을 추가해줘.
command 는 "npm test 1>&2 ¦¦ exit 2" 로. 기존 env, permissions 설정은 유지해줘.
```

👀 **확인할 것**: 설정 파일에 이 부분이 들어 있어야 함

```
"hooks": {
  "TaskCompleted": [
    { "hooks": [ { "type": "command", "command": "npm test 1>&2 ¦¦ exit 2" } ] }
  ]
}
```

💬 **강사 질문**: "`|| exit 2` 에서 2 대신 1 이면?" → 막지 못함. **2 여야 막는다**

6-2. 🔥 일부러 걸려보기 (20분)

💡 **왜 하나요?** 검문소가 진짜로 막는지 보려면 **일부러 실패하는 상황**을 만들어야 합니다. 그래서 팀원에게 "**아직 코드가 지원하지 않는 기능의 테스트**" 를 먼저 쓰게 하고 완료를 시도하게 합니다.

💬 **클로드 대화창**

```
agent team 으로 팀원 1명(tester)을 뽑아줘, 모델은 Sonnet.
할 일: test/books.test.js 에 "검색어 앞뒤에 공백이 있어도 결과는 같다" 테스트를 추가해
(예: searchBooks(' 리팩터링 ') 의 결과가 1권). src/books.js 는 고치지 말고 테스트만 추가한 다음 할 일을 완료로 표시해.
```

👀 **확인할 것**: 테스트가 실패하므로 **완료 표시가 거부**되고, tester 가 실패 메시지를 받는 것

이어서 💬

```
tester 에게 전달해줘: 완료가 거부된 이유를 읽고, src/books.js 의 searchBooks 를 고쳐서 테스트를 통과시킨 뒤 다시 완료해.
```

👀 **확인할 것**: 이번엔 완료 ✅

6-3. 정리 (10분)

💬 `팀원 퇴근시켜줘.`

⌨️ **터미널**

```
git add . && git commit -m "chore: TaskCompleted 검문소 + 검색 공백 테스트"   # 검문소 설정과 새 테스트를 세이브
```

✅ 체크포인트

- [ ] 완료 표시가 **거부**되는 걸 봤다
- [ ] 고친 뒤 완료가 **통과**됐다
- [ ] `exit 2` 의 뜻을 말할 수 있다

⭐ 보너스: **TeammateIdle** hook (팀원이 **쉬려고 할 때** 실행) 도 달아보기 — `git status --porcelain` (커밋 안 한 파일 목록을 짧게 출력) 에 결과가 있으면 exit 2 → "커밋하고 쉬어"

🆘 따라잡기: 6-1 의 JSON 을 설정 파일에 직접 붙여넣고 6-2 만 실행

☕ 휴식 (10분)

블록 7

🕵️ 디버깅 토론 — 짝꿍이 심은 버그 찾기 (30분)

목표

원인을 모르는 버그를 **경쟁 가설 토론**으로 찾기

> 💡 **이 블록을 하는 이유**: 클로드 한 명은 **처음 찾은 그럴듯한 원인에 꽂혀서** 거기서 멈추는 버릇이 있습니다 (닻 내림 효과). 팀원 여럿이 **서로 다른 원인을 주장하고 반박**하면, 반박에서 살아남은 원인이 진짜일 가능성이 높습니다.

📖 용어

말	뜻
가설	"원인은 아마 ○○일 것이다" 라는 추측
경쟁 가설	서로 **다른** 가설을 각자 맡아서 증명·반박
증상	사용자가 겪는 이상한 결과 ("반납했는데 권수가 안 늘어나요")

7-1. 버그 심기 (8분, 짝꿍과 자리 바꾸기)

💡 **왜 짝꿍이 심나요?** 내가 심으면 답을 알아서 토론이 의미가 없습니다.

A 가 B 의 컴퓨터에서 ⌨️ 새 터미널을 열고

```
cd ~/projects/library-app && claude   # 짝꿍 앱 폴더에서 새 클로드 대화 시작
```

💬 **클로드 대화창**

```
이 앱에 미묘한 버그를 하나만 몰래 심어줘. 조건:
- 서버는 정상으로 켜지고, 화면에서 사용하다 보면 이상한 결과가 나오는 종류
- 주석이나 이름으로 티 내지 말 것
- 바꾼 내용은 나에게만 말해줘. 그리고 git add . && git commit -m "chore: 정리" 로 커밋해.
```

→ A 는 **증상 카드**만 종이에 적어서 B 에게 줌 (예: "반납했는데 남은 권수가 안 늘어나요") → 💬 `/exit` → 역할 바꿔서 B 도 A 의 컴퓨터에 버그 심기

7-2. 🔬 가설 토론 팀 (17분)

💡 **왜 git 기록을 금지하나요?** `git log` · `git diff` (📖 바뀐 내용 비교) · `git show` 를 보면 짝꿍이 바꾼 줄이 바로 보여서 **토론 없이 답을 알게** 됩니다. 오늘은 토론 연습이 목적입니다.

💬 **클로드 대화창** (내 컴퓨터, 팀 켜진 대화)

```
사용자 증상: (증상 카드 내용)
agent team 으로 팀원 3명(모델 Sonnet)을 뽑아서 각자 다른 가설을 조사하게 해줘.
과학 토론처럼 서로의 가설을 반박하려고 노력하게 하고, git 기록은 보지 말고 코드와 실행으로만 증명해.
합의된 원인과 증거(재현 명령어)를 docs/findings.md 에 정리해줘. 아직 고치지는 마.
```

→ 시간 재기 ⏱️ → 찾으면 짝꿍에게 확인 → 맞으면 🎉

7-3. 정리 (5분)

💬 **클로드 대화창**

```
findings.md 의 원인대로 고쳐줘. 그리고 같은 버그가 다시 생기지 않게 테스트도 하나 추가해줘.
```

⌨️ **터미널**

```
npm test && git add . && git commit -m "fix: 디버깅 토론으로 찾은 버그 수정"   # 테스트가 통과할 때만(&&) 담아서 세이브
```

✅ 체크포인트

- [ ] 팀원들이 서로의 가설을 반박하는 메시지를 봤다
- [ ] `docs/findings.md` 에 원인과 재현 명령어가 있다
- [ ] 짝꿍이 심은 버그를 찾았다 (못 찾았으면 몇 분 걸렸는지라도 기록)

☕ 휴식 (10분)

휴식 중: 블록 8 메뉴판을 보고 **고를 상황 하나** 정해 오기

블록 8

🚀 나만의 팀 — 2바퀴 (90분)

목표

강사 없이 내가 팀장이 되어, **팀이 필요한지 판단하는 것부터** 퇴근까지 직접

> 💡 **이 블록을 하는 이유**: 1바퀴는 "따라 하기" 였습니다. 실제로 쓸 때는 **"이 일에 팀이 필요한가?"** 부터 내가 정해야 합니다. 그 판단을 혼자 해보는 게 오늘의 마무리입니다.

8-1. 고르기 + 팀 설계서 (20분)

난이도	상황	팀 구성 힌트
⭐	도서 앱 **README·API 문서 리뷰**	초보 사용자 / 개발자 / 깐깐한 편집자
⭐	**기술 조사**: "이 앱을 SQLite(📖 파일 하나로 된 작은 데이터베이스)로 옮기면?"	찬성 / 반대 / 옮기는 방법 설계
⭐⭐	도서 앱에 **관리자 기능**(책 추가·삭제)	books / auth / web, 파일 주인
⭐⭐	도서 앱 **테스트 늘리기**	모듈별 테스트 담당 + 블록 6 검문소
⭐⭐	**이전 수업 프로젝트**(메모 앱 등) 보안·성능·테스트 리뷰	블록 3 역할 카드 재사용
⭐⭐⭐	**내 프로젝트**에 새 기능	직접 설계, 파일 소유표 필수

팀 설계서 (종이나 `docs/team-design.md`) — 강사 확인 후 시작

```
① 왜 팀이어야 하나? (혼자·심부름꾼으로 안 되는 이유 한 줄)
② 팀원: 이름 / 역할 / 모델 / 맡을 파일
③ 할 일 보드 초안: 팀원당 4~6개, 선행 작업
④ 결과물: 어떤 파일에 무엇이 남는가
⑤ 안전장치: 시작 전 커밋, 검문소(hook) 필요 여부
```

> 💡 **①이 제일 중요한 이유**: ①에 답을 못 하면 그 일은 혼자나 심부름꾼으로 하는 게 더 싸고 빠릅니다.

8-2. 가동 (55분)

- 💬 업무 지시서 먼저 보여달라고 → **내가 확인** → 팀원 뽑기 (블록 5-2 와 같은 순서)
- 10분마다 **현황판(↑↓ Enter)·할 일 보드(Ctrl + T)** 들여다보기
- 끝난 팀원은 **바로 퇴근**
- 💬 `/usage` 로 시작 전·후 사용량 메모

8-3. 회고 메모 (15분)

1. 팀이 정말 필요했나? (다시 한다면 혼자 / 심부름꾼 / 팀?)
2. 팀원이 엇나간 순간과 어떻게 바로잡았나
3. 사용량은 얼마나 썼고, 그만한 가치가 있었나

👩‍🏫 강사 운영 팁

- 자리를 돌며 **"① 왜 팀이어야 해요?"** 부터 묻기. 답이 약하면 ⭐ 로 유도
- 마우스·키보드 잡지 않기. **"지금 팀장이 뭘 기다리고 있어요?"** 만 묻기
- 사용량 한도에 걸린 학생 → 짝꿍 화면에서 같이 (설계서는 각자 발표)

✅ 체크포인트

- [ ] 팀 설계서 ① 에 답했다
- [ ] 팀원 업무 지시서를 먼저 확인했다
- [ ] 결과물 파일이 있고, 팀원을 모두 퇴근시켰다

블록 9

발표 & 회고 (30분)

학생 5~6명이 **화면 공유**하며 2분씩:

1. 팀 설계서 ① **"왜 팀이어야 했나"**
2. 할 일 보드 / 결과물 보여주기
3. 팀원이 엇나간 순간 하나 + 내가 어떻게 끼어들었나
4. 사용량 대비 가치 — **다음엔 혼자 할 것 같은 일**은?

🏁 최종 체크리스트

블록	한 것	배운 것
1	심부름꾼 (Explore, 역할 카드, @, Ctrl+B)	팀 전에 심부름꾼, 도구로 권한 막기
2	팀 켜기, 첫 팀, 현황판	↑↓ Enter Esc, Ctrl+T, 퇴근
3	혼자 vs 팀 리뷰	관점 나누기, 역할 카드 재사용, 비용 비교
4	반박 회의, 직접 대화	팀원끼리 메신저, 거짓 경보 걸러내기
5	구현 팀	파일 주인, 계획 먼저, 지시서 확인
6	TaskCompleted hook	exit 2 로 완료 막기
7	경쟁 가설 토론	닻 내림 피하기
8	나만의 팀	"왜 팀이어야 하나"

오늘의 한 줄: **"팀은 회의가 필요한 큰일에만. 지시서는 내가 먼저 읽고, 파일 주인을 정하고, 끝나면 퇴근시킨다."**

부록 A

시간 조절

시간 모자랄 때 자르는 순서

1. 블록 1-4 (백그라운드) → 강사 시연
2. 🧩 오전 퀴즈 → 퀴즈 1만
3. 블록 4-2 (직접 끼어들기) → 4-1 중에 한 번만
4. 블록 5 → 예약 기능 빼고 **심각도 높음 수리만**
5. 블록 7 → 버그 심기 대신 부록 C 의 L1 증상("재고 0인데 대출됨")으로 토론

절대 자르면 안 되는 것: 블록 3 혼자 vs 팀 비교, 블록 5-2 업무 지시서 먼저 확인

시간이 남을 때

1. **분할 화면**: mac + tmux 학생은 ⌨️ `claude --teammate-mode auto` 로 블록 3 다시
2. **TeammateIdle 검문소**: 커밋 안 한 파일이 있으면 못 쉬게 (블록 6 보너스)
3. **worktree 비교**: ⌨️ `claude --worktree fix-auth` 와 `claude --worktree fix-books` 를 두 터미널에 띄워 **내가 직접** 두 세션 운영 (📖 worktree = 같은 저장소의 작업 폴더 복사본. 파일이 안 겹침) → 팀과 뭐가 다른지 토론
4. **역할 카드 공유**: `.claude/agents/` 를 짝꿍 저장소에 PR 로 보내기 (Git 강의 연결)

부록 B

자주 막히는 곳

증상	해결
`git clone` 에서 `repository not found`	폴더 주소(`/tree/main/...`)를 넣음 → `https://github.com/NujabesCode/library-app.git`
`git clone` 에서 `Too many arguments`	주소 뒤에 단어가 2개 이상 → 주소만
💬 문장을 터미널에 붙여넣어 에러	💬 는 `claude` 실행 **후** 대화창에
팀원이 안 생기고 심부름꾼만 생김	설정 확인 (`.claude/settings.json` 의 env) → `/exit` 후 다시 `claude` → "**agent team 으로**"
`claude -p` **로 실행했더니 팀이 안 생김**	팀은 **대화형 세션에서만** (그냥 `claude`)
현황판이 안 보임	입력창 **아래**. 모두 쉬면 30초 뒤 숨김 → 이름으로 메시지
팀원 화면에서 Esc → 팀원이 멈춤	팀원 화면의 Esc = 그 팀원 작업 중단. "계속해" 라고 말하기
팀장이 직접 코드를 씀	💬 `팀원들이 끝날 때까지 기다려. 직접 구현하지 마`
팀장이 일이 남았는데 끝났다고 함	💬 `할 일 보드에 남은 일이 있어. 계속 진행해`
할 일이 계속 "진행 중"	실제로 끝났는지 확인 → 💬 `○번 일을 완료로 바꿔줘`
허락 요청이 계속 뜸	`permissions.allow` 에 `Bash(npm test:*)` 등록 (블록 5-1)

증상	해결
두 팀원이 같은 파일을 덮어씀	파일 소유표를 지시서에 명시. ⌨️ `git diff` 로 확인 후 복구
hook 이 완료를 안 막음	명령이 `exit 2` 로 끝나는지 확인 (1 이면 못 막음)
모든 할 일이 완료 거부됨	테스트가 원래 깨져 있음 → 먼저 `npm test` 를 통과 상태로
`/resume` **했더니 팀원이 없음**	알려진 한계. 💬 "새 팀원을 뽑아줘"
사용량 한도 도달	팀원 수 줄이기, Sonnet, 끝난 팀원 퇴근 → 그래도 안 되면 짝꿍 화면으로
분할 화면이 안 됨	Windows Terminal·VS Code 터미널은 미지원 → 기본(한 화면) 모드

부록 C

🔒 강사용 정답지 — 연습용 앱에 심어둔 문제

> 블록 3-4 에서 공개. 학생 리뷰가 아래와 다른 **진짜 문제**를 찾으면 가산점.

보안 (S)

#	파일	문제	재현
S1	`src/auth.js`	비밀번호를 **평문**으로 저장	코드 확인
S2	`src/auth.js` `signup`	같은 아이디로 가입하면 **기존 계정 비밀번호를 덮어씀** (계정 탈취)	`hong` 으로 두 번 가입 → 두 번째 비밀번호로 로그인됨
S3	`src/auth.js` `login`	`users[username] == password` → **없는 아이디 + 비밀번호 생략** 시 `undefined == undefined` 로 로그인 성공	`curl -XPOST localhost:3000/api/login -d '{"username":"nobody"}'` → 토큰 발급
S4	`src/auth.js` `verify`	토큰 = 아이디의 base64. 서명이 없어 **누구나 위조**	`Authorization: $(echo -n hacker | base64)` 로 대출 성공
S5	`server.js` 정적 파일	`path.join('public', req.url)` → **경로 탈출**로 서버 소스 노출	`curl --path-as-is http://localhost:3000/../server.js`
S6	`public/index.html`	책 제목·저자를 `innerHTML` 로 출력 → **XSS** 위험 (데이터에 HTML 이 들어오면 실행)	코드 확인

성능·안정성 (P)

#	파일	문제	재현
P1	`server.js` `readBody`	`JSON.parse` 에 try/catch 없음 → **잘못된 JSON 한 번에 서버 전체 다운**	`curl -XPOST localhost:3000/api/login -d '{bad'` → 서버 종료
P2	`src/books.js`	요청마다 `readFileSync` / `writeFileSync` 로 파일 전체를 **동기** 읽기·쓰기	코드 확인
P3	`src/books.js` `searchBooks`	이중 반복문 + 여러 단어가 맞으면 **같은 책 중복** 반환	`/api/books?q=자바스크립트 가이드` → 3번 책이 2번
P4	`src/books.js` `loans`	대출 기록이 **메모리에만** → 서버 재시작하면 반납 불가	대출 → 서버 재시작 → 반납

로직·테스트 (L, T)

#	파일	문제	재현
L1	`src/books.js` `borrowBook`	`available >= 0` → **재고 0인데 대출**되어 -1 까지 내려감	2번 책(1권) 두 번 대출 → `available: -1`
L2	`src/books.js` `lateFee`	일찍 반납하면 **음수 연체료** (2일 만에 반납 → -1200원)	`lateFee(1월 1일, 1월 3일)`
T1	`test/`	정상 경로 테스트 2개뿐. 재고 0 대출, 반납, 연체료, 인증 테스트 **없음**	`npm test` 결과 확인

채점 기준: 12개 중 몇 개를 찾았나 + 재현까지 한 것은 ×2. 결과는 반마다 다르니 **혼자 / 팀 평균을 기록**해두면 다음 수업에서 근거로 쓸 수 있습니다.

> 💡 검색어에 공백이 섞이면 모든 책이 중복으로 나오는 문제(P3 의 변형)는 블록 6 에서 일부러 씁니다. 블록 5 에서 이미 고쳐졌다면 블록 6 의 테스트 주제를 아직 안 고친 다른 문제로 바꾸세요.